Failure Propagation in Multi-Agent LLM Systems: Characterization and Containment with Circuit Breakers

Goutham Patley
Independent Researcher
gp4em@virginia.edu

Abstract

Large language models (LLMs) are increasingly deployed as compositional systems in which multiple agents interact to solve complex tasks. While such architectures enable modularity and improved capability, they introduce a new class of failure modes: errors originating in one component can propagate through the system, leading to cascade failures.

In this work, we study failure propagation in multi-agent LLM pipelines and evaluate circuit breakers as a mechanism for containment. We formalize cascade failures through a probabilistic model of upstream error propagation and introduce metrics such as Cascade Failure Rate (CFR) to quantify system behavior.

We conduct a large-scale simulation study (100,000+ trials) across varying pipeline depths and failure conditions, demonstrating that circuit breaker mechanisms reduce cascade failures by 50–89% with strong statistical significance. We complement these results with real-world API-based experiments using a planner–executor–verifier pipeline, where we observe that up to 62% of upstream failures propagate to final task failure without protection. Circuit breakers eliminate cascade failures entirely in this setting and, under moderate perturbation, improve end-to-end success rates.

We further analyze adaptive circuit breaker strategies and find that naive adaptive policies can over-trigger, highlighting the challenges of designing robust control mechanisms in LLM systems. Our results suggest that failure containment is a critical systems concern for compositional AI and that circuit breaker-inspired mechanisms provide a promising direction for improving reliability.

1 Introduction

Large language models (LLMs) are increasingly used as building blocks in compositional systems, where multiple agents interact through structured interfaces to perform complex tasks. These systems include planner–executor pipelines, tool-augmented agents, and multi-agent reasoning frameworks. While such architectures enable modularity and specialization, they also introduce new system-level failure modes.

A key challenge in these systems is failure propagation: errors originating in upstream components can propagate through downstream stages, leading to cascade failures. For example, an incorrect plan generated by a planning agent may cause an execution agent to produce invalid outputs, which in turn leads to failure in verification. Unlike isolated errors, these failures compound across stages and are harder to detect and mitigate.

Despite the growing adoption of multi-agent LLM systems, failure propagation remains poorly understood. Most prior work focuses on improving individual model performance or prompt design, with limited attention to system-level reliability and error containment.

We frame compositional LLM systems as reliability-critical distributed systems, where failure propagation—not just individual model error—becomes the dominant challenge. In this work, we study failure propagation as a first-class systems phenomenon in compositional LLM pipelines. We draw inspiration from distributed systems, where circuit breakers are used to prevent cascading failures by halting requests to failing components. We adapt this concept to LLM-based agents and evaluate its effectiveness in both simulated and real-world settings.

Contributions This paper makes the following contributions:

- We formalize failure propagation in compositional LLM systems and introduce Cascade Failure Rate (CFR) as a metric for quantifying error amplification across agent pipelines.
- We adapt circuit breaker mechanisms from distributed systems to LLM-based agents, using inferred signals such as output validity and confidence to detect and contain failures.
- We provide both large-scale simulation (100,000+ trials) and real-world API validation, showing that circuit breakers can eliminate cascade failures (CFR: $0.62 \rightarrow 0.00$) in multi-agent pipelines.
- We identify a key tradeoff between failure containment and task success, and show that naive adaptive policies can over-trigger, highlighting the challenges of adaptive control in LLM systems.

2 Theoretical Framework

We model a compositional LLM system as a sequence of agents, where each stage transforms an input and may introduce or propagate errors.

Let p_u denote the probability of an upstream failure and p_p denote the probability that a failure propagates to downstream components. The probability of final failure can be expressed as:

$$P(\text{final failure}) = p_u \times p_p \tag{1}$$

We define Cascade Failure Rate (CFR) as:

$$\text{CFR} = P(\text{final failure} \mid \text{upstream failure}) \tag{2}$$

CFR captures the likelihood that an upstream error propagates through the system. As pipeline depth increases, the likelihood of error amplification grows due to repeated propagation across stages. Circuit breakers reduce CFR by intercepting faulty states and preventing further propagation, effectively reducing the propagation probability p_p.

3 Method

We evaluate failure propagation and containment in a multi-agent pipeline under three configurations:

- **No Circuit Breaker**: Agents execute sequentially without intervention, allowing errors to propagate freely.
- **AI-Based Circuit Breaker**: Each stage evaluates upstream outputs using structural and confidence-based signals (e.g., JSON validity, completeness, and confidence scores). If a failure is detected, execution is halted or fallback logic is triggered.

- **Adaptive Circuit Breaker**: The intervention threshold is dynamically adjusted based on observed failure patterns and recent system behavior, allowing more aggressive containment under high-error conditions.

3.1 Simulation Setup

We simulate multi-agent pipelines of varying depths (2–6 stages) with probabilistic failure injection. Each agent introduces upstream failures with probability p_u, and propagation occurs with probability p_p. We run over 100,000 trials across configurations to estimate Cascade Failure Rate (CFR).

3.2 Real-World Setup

We implement a planner–executor–verifier pipeline using real LLM APIs. Failures are injected via controlled perturbations of planner outputs at a fixed rate, including structured corruption of intermediate artifacts. Each configuration is evaluated across multiple runs, and metrics such as upstream failure rate, cascade rate, final success rate, and breaker activation rate are recorded.

4 Simulation Results

We evaluate cascade failure behavior across varying pipeline depths under different circuit breaker configurations. Figure 1 shows Cascade Failure Rate (CFR) as a function of chain length.

Without circuit breakers, CFR increases sharply with depth, exceeding 0.6 at chain length 6. This indicates strong error amplification as failures propagate across multiple stages.

A simple circuit breaker reduces CFR moderately but still exhibits increasing failure rates with depth. In contrast, the AI-based circuit breaker significantly suppresses propagation, maintaining CFR below 0.3 across all chain lengths.

The adaptive circuit breaker achieves the strongest containment, reducing CFR consistently as depth increases. Notably, CFR decreases with depth under adaptive control, suggesting that aggressive intervention can effectively halt failure cascades.

These results demonstrate that failure propagation scales with compositional depth and that circuit breaker mechanisms provide effective containment.

5 Real-World API Validation

5.1 Setup

We implement a three-agent pipeline consisting of a planner, executor, and verifier using a commercial LLM API. Each task consists of a question with a known reference answer. The planner produces a structured plan, the executor generates an answer, and the verifier evaluates correctness.

To simulate realistic failure conditions, we inject perturbations into the planner output with a specified probability. We use a graded corruption mechanism that introduces mild, moderate, and severe perturbations to mimic degradation in upstream outputs.

We evaluate three configurations: no circuit breaker, AI-based circuit breaker, and adaptive circuit breaker.

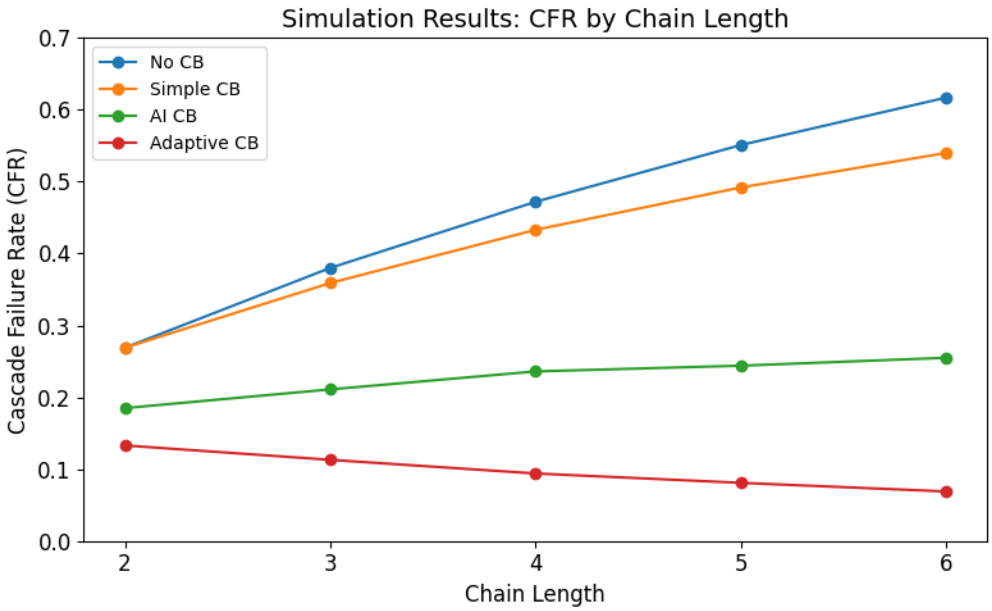

Figure 1: Simulation results across chain lengths 2–6. Baseline cascade failure rate increases with chain depth, indicating compounding error propagation. Circuit breaker mechanisms substantially reduce cascade failures, with adaptive strategies achieving the lowest propagation rates in longer chains.

5.2 Results

Figure 2 shows the impact of circuit breakers under real API execution.

Without circuit breakers, 62% of upstream failures propagate to final task failure, resulting in a final success rate of 0.38. This confirms that even moderate upstream errors can significantly degrade end-to-end performance.

The AI-based circuit breaker eliminates cascade failures entirely (cascade rate = 0.00) and improves final success rate to 0.44 under moderate perturbation. This demonstrates that early detection and containment can improve overall system reliability.

The adaptive circuit breaker also eliminates cascade failures but achieves a slightly lower success rate (0.40), indicating over-triggering and premature intervention.

These results highlight a key tradeoff: while aggressive containment eliminates cascades, it may also reduce task completion due to false positives.

6 Tradeoff Analysis

Figure 3 illustrates the containment–intervention tradeoff between the two breaker variants used in real API evaluation.

Both AI-based and adaptive circuit breakers eliminate cascade failures in this regime. However, they differ in how aggressively they intervene. The adaptive breaker triggers more frequently, while also achieving a lower final success rate. This suggests that its policy is over-sensitive under the evaluated conditions.

By contrast, the static AI-based breaker achieves a better balance: it maintains zero cascade failures while preserving a higher end-to-end success rate. This result highlights that containment

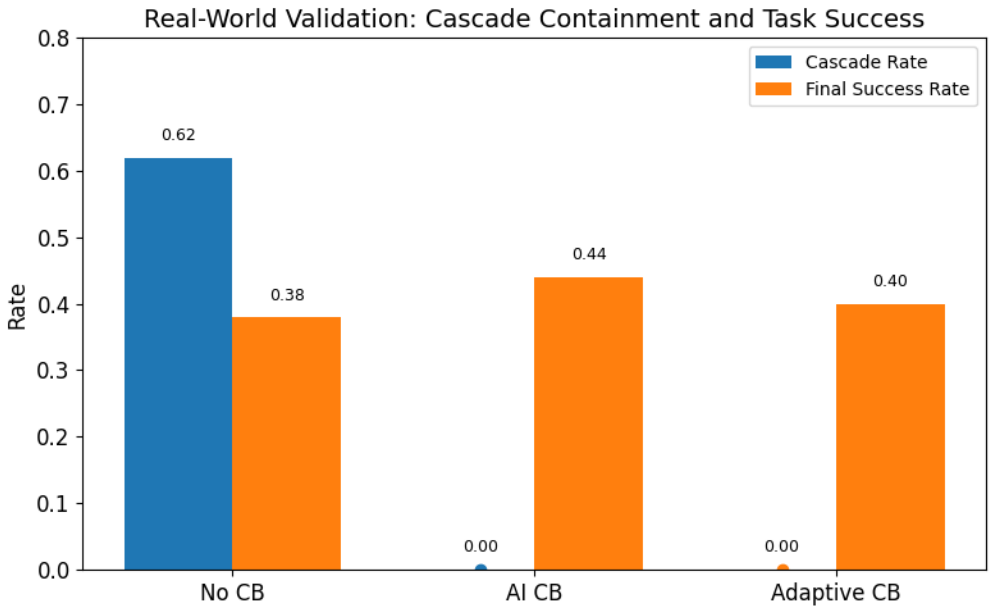

Figure 2: Real-world validation under moderate perturbation (inject rate = 0.2). Without protection, 62% of upstream failures propagate to final failure. Both AI-based and adaptive circuit breakers eliminate cascade failures (CFR = 0.0). The AI-based circuit breaker achieves the highest end-to-end success rate in this setting.

quality alone is not sufficient; intervention efficiency also matters.

7 Discussion

Our results show that failure propagation is a dominant challenge in compositional LLM systems, particularly as pipeline depth increases. Without containment, even modest upstream error rates can lead to high cascade failure rates.

Circuit breaker mechanisms effectively eliminate cascade failures by interrupting propagation. However, this introduces a tradeoff: aggressive intervention can reduce final task success by prematurely halting recoverable executions.

The performance of the adaptive circuit breaker highlights this challenge. While it achieves strong containment, it can over-trigger, suggesting that designing optimal intervention policies remains a key open problem.

More broadly, the results suggest that techniques from distributed systems can be meaningfully adapted to compositional AI systems. The key insight is not simply that individual LLM outputs can be wrong, but that errors can accumulate and amplify across stages if left unchecked.

8 Limitations

Our study has several limitations. First, we evaluate relatively simple multi-agent pipelines, which may not capture the full complexity of production systems. Second, failure injection is synthetic and may not fully reflect real-world error distributions.

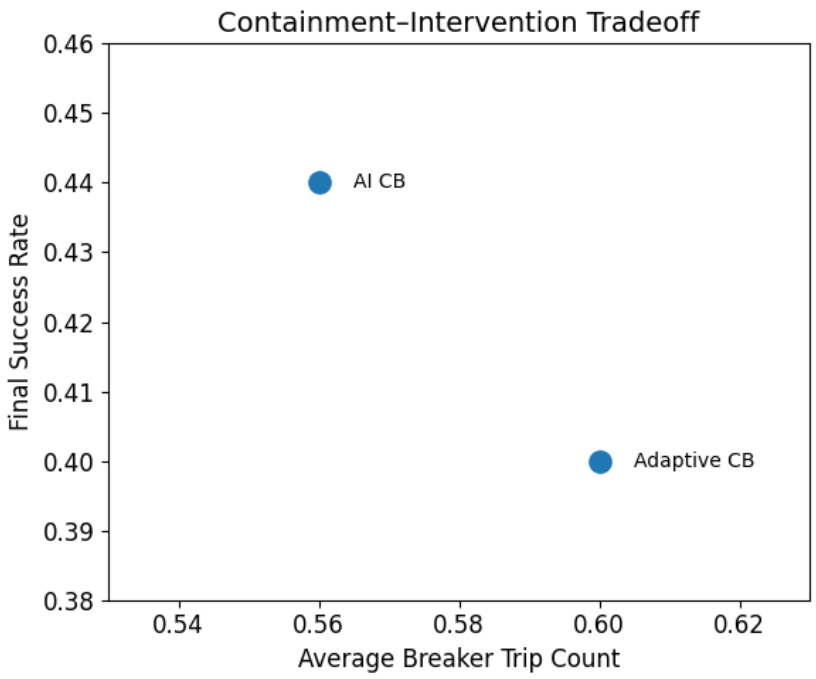

Figure 3: Containment–intervention tradeoff. While both circuit breaker variants eliminate cascade failures, adaptive policies trigger more frequently, resulting in lower end-to-end success compared to the static AI-based breaker. This highlights the challenge of designing efficient adaptive control mechanisms.

Third, our adaptive circuit breaker is heuristic and not optimized, and more principled approaches may yield better performance. Finally, our evaluation focuses on system-level metrics and does not deeply analyze semantic correctness or user-level impact.

9 Future Work

Future work should explore principled adaptive circuit breaker policies that balance containment and task success. Extending failure detection to capture semantic inconsistencies and reasoning errors is another key direction.

Evaluating these mechanisms in more complex multi-agent systems and integrating them with existing LLM reliability techniques may further improve robustness.

10 Related Work

10.1 Multi-Agent and Compositional LLM Systems

Recent work has explored the use of large language models (LLMs) as components in compositional systems, where multiple agents interact to solve complex tasks. These include planner–executor pipelines, tool-augmented agents, and autonomous agent frameworks such as AutoGPT [2] and BabyAGI [4]. More recent work on LLM-based agents has further explored tool use and structured reasoning in multi-step workflows [7, 11].

While these systems improve modularity and flexibility, most prior work focuses on improving task performance through prompt engineering, agent coordination, or tool integration. Relatively little attention has been paid to system-level reliability and error propagation across agent boundaries. In contrast, our work explicitly studies failure propagation as a systems phenomenon.

10.2 Reliability and Robustness in LLMs

A growing body of work has examined reliability issues in LLMs, including hallucinations, reasoning errors, and sensitivity to prompting. Techniques such as chain-of-thought prompting [10], self-consistency decoding [9], and verification-based approaches aim to improve reasoning accuracy.

Retrieval-augmented generation (RAG) [3] and tool use [7] have also been proposed to improve factual correctness. However, these methods primarily target individual model outputs rather than system-level behavior. Our work complements this literature by analyzing how errors propagate across multiple stages in compositional systems.

10.3 Error Propagation in Machine Learning Pipelines

Error propagation has been studied in traditional machine learning pipelines and structured prediction systems, where inaccuracies in early stages can affect downstream components [8]. These studies highlight how multi-stage architectures can amplify small errors.

We extend this perspective to LLM-based systems, where intermediate representations are unstructured and probabilistic. This increases the risk of cascade failures, as downstream agents often rely heavily on upstream outputs without explicit validation.

10.4 Fault Tolerance and Circuit Breakers in Distributed Systems

In distributed systems, circuit breakers are widely used to prevent cascading failures by halting requests to failing services [5, 6]. Systems such as Netflix Hystrix [5] implement mechanisms for failure detection, isolation, and recovery.

We draw inspiration from this literature and adapt circuit breaker concepts to LLM-based agents. Unlike traditional services, LLM outputs are probabilistic and lack explicit failure signals, requiring the use of inferred indicators such as confidence and output validity.

10.5 Adaptive Control and Dynamic Policies

Adaptive control strategies have been explored in distributed systems and reinforcement learning, where policies dynamically adjust based on observed outcomes [1]. However, adaptive reliability mechanisms for LLM-based systems remain underexplored.

Our results show that naive adaptive circuit breaker policies can over-trigger, leading to reduced performance despite improved containment. This highlights the need for more principled adaptive control strategies in compositional AI systems.

References

[1] Karl Johan Å ström and Björn Wittenmark. *Adaptive Control: Stability, Convergence, and Robustness*. Addison-Wesley, 1995.

[2] Significant Gravitas. Autogpt: An autonomous gpt-4 experiment. `https://github.com/Significant-Gravitas/AutoGPT`, 2023.

[3] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.

[4] Yohei Nakajima. Babyagi: Task-driven autonomous agent. `https://github.com/yoheinakajima/babyagi`, 2023.

[5] Netflix OSS. Hystrix: Latency and fault tolerance for distributed systems. `https://github.com/Netflix/Hystrix`, 2012.

[6] Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007.

[7] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

[8] D. Sculley et al. Understanding error propagation in machine learning pipelines. *NIPS Workshop*, 2015.

[9] Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[10] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

[11] Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.